CONTRACT-HEADHUNTER · JOBMATCH

Jobmatch Runbook

How to set up the repo, what runs today, how a posting travels from intake to a reviewed match, and what to do when you have new data. As of 2026-10-19, build phases 1 and 2 are done; everything after normalization is still planned.

What exists today

BUILT AND TESTED

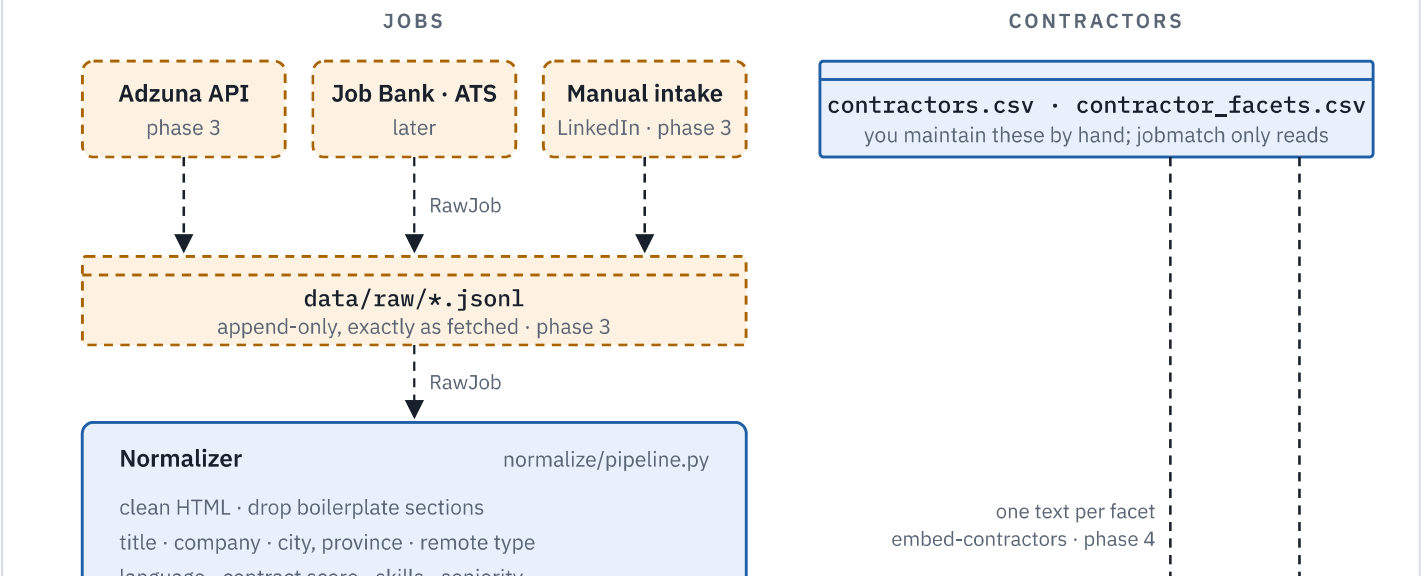
- Config loading and pydantic schemas
- CSV store and NumPy vector store, with locks and backups
- `validate-data` and `smoke-embed` commands
- Embedding model loads on CPU (dev PC)
- Normalizer: raw posting to clean, filtered `Job`

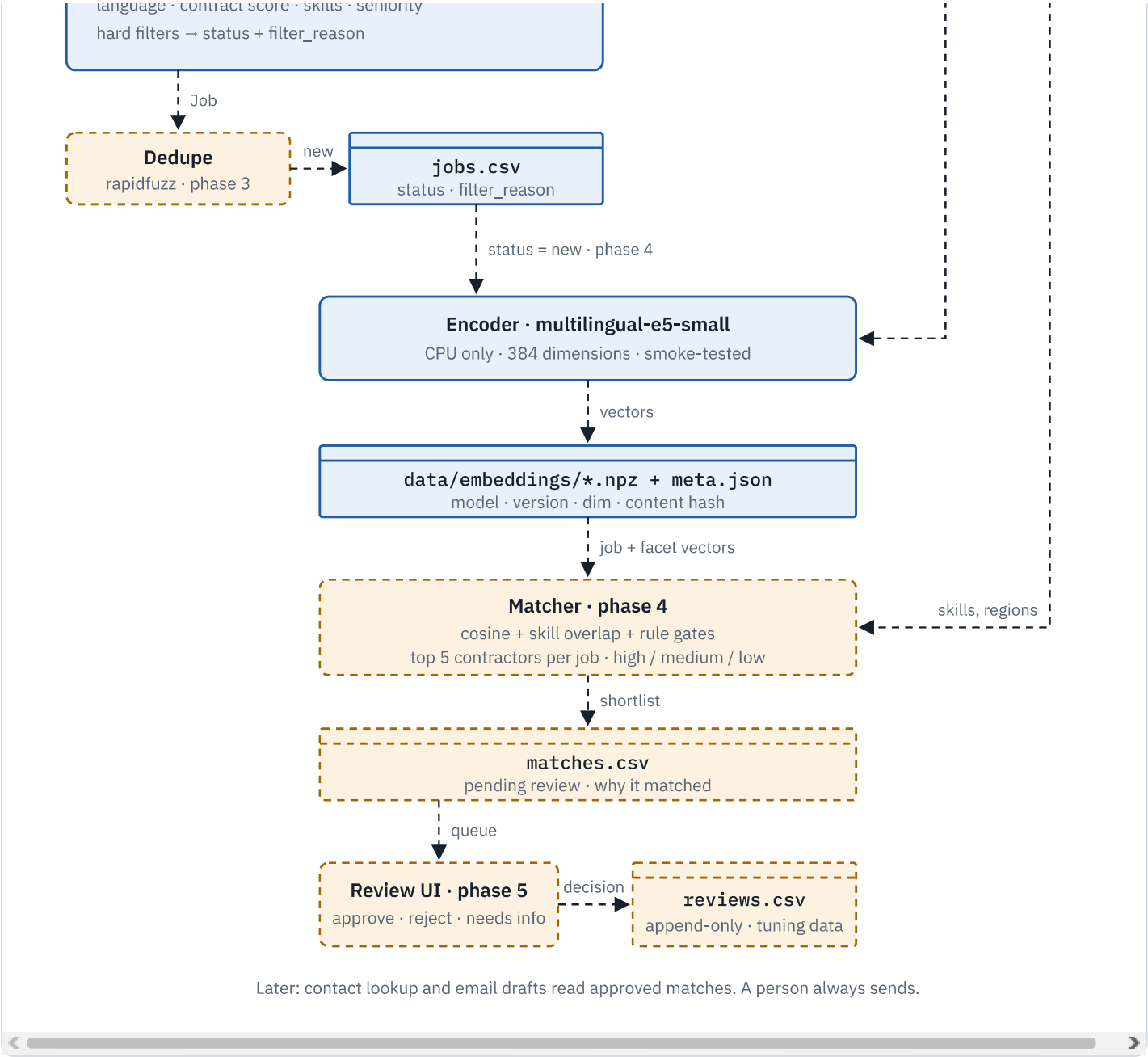
PLANNED

- Phase 3: Adzuna source, manual intake, raw store, dedupe
- Phase 4: embed jobs and facets, matcher, evaluation
- Phase 5: Streamlit review UI
- Phase 6: `run-once` on a schedule (launchd)

How the data flows

Built and tested · Planned (phase noted) · Band on top: a file on disk under `data/`





Everything flows down the jobs lane; contractors join at the encoder (their facet text) and at the matcher (their skills and regions for the rule gates). Reviewer decisions in reviews.csv are the data used to tune the thresholds in settings.yaml.

Each stage in plain terms

Ingestion

PHASE 3

Each source turns a posting into a `RawJob` : title, company, location, description (HTML or text), and the source's own employment type if it has one. LinkedIn jobs only arrive by a person pasting them in. Raw records are kept untouched in `data/raw/` so the rules can be re-run later.

Processing

Processing

☐ BUILT

The normalizer cleans the text, drops "About us", benefits and equal-opportunity sections, and pulls out requirements, title, company, city and province, remote type, language (en, fr or bilingual), contract signals and duration, skills and seniority.

Hard filters favour recall. A posting is dropped only when it is clearly outside Canada, clearly permanent, older than 30 days, or long and in neither English nor French. Every drop records its `filter_reason`. Postings without a description become `needs_details`.

Matching

⬚ PHASE 4

Each new job and each contractor facet is turned into a vector with the same text template. The matcher scores every facet against the job (cosine similarity plus skill overlap), applies rule gates such as must-have skills, language and region, keeps the top 5 contractors, and assigns a confidence band. Raw cosine scores bunch together, so phase 4 rescales them before banding.

Review

⬚ PHASE 5

A person works through the queue and approves, rejects with a reason code, or asks for more information. The system never acts on a match by itself.

Get it running

Run every command from the repo folder. They all run inside the Docker container, so nothing is installed on the machine itself.

1 **Install Docker Desktop**

On the client Mac, first check that Docker Desktop supports its macOS version. In Docker's settings, cap memory at about 4 GB.

2 **Create your `.env`**

```
cp .env.example .env
```

Keep `HF_HUB_OFFLINE=0` until the model has been downloaded once (step 4).

3 **Build the image**

```
docker compose build
```

About 460 MB to download the first time; 2.1 GB on disk.

4 **Download and check the embedding model**

```
docker compose run --rm app python -m jobmatch smoke-embed
```

One-time download of about 500 MB. It prints load time, milliseconds per text and peak memory; run it on the client Mac to get real numbers. Then set `HF_HUB_OFFLINE=1` in `.env`.

5 Check the code and the data

```
docker compose run --rm app pytest
docker compose run --rm app python -m jobmatch validate-data --data-dir data/samples
docker compose run --rm app python -m jobmatch validate-data
```

The last command checks your real files in `data/`.

Commands

COMMAND	WHAT IT DOES	STATUS
`python -m jobmatch validate-data`	Checks config files, every CSV (headers, each row, duplicate IDs, facets pointing at unknown contractors) and the embedding files	☐ WORKS
`python -m jobmatch smoke-embed`	Loads the model and checks English and French similarity on sample texts	☐ WORKS
`pytest · ruff check .`	Tests and lint	☐ WORKS
`python -m jobmatch ingest-manual <file>`	Import manually collected jobs	⬚ PHASE 3
`python -m jobmatch embed-contractors`	Embed contractor facets, only those that changed	⬚ PHASE 4
`python -m jobmatch run-once`	Fetch, normalize, embed, match, store	⬚ PHASE 6

Prefix each with `docker compose run --rm app`. Commands that are not built yet print the phase they belong to and exit.

Try the normalizer on a posting of your own

There is no import command until phase 3, but you can feed one posting to the normalizer directly:

```
docker compose run --rm -T app python - <<'EOF'
from datetime import UTC, datetime
```

```
from jobmatch.config import load_settings
from jobmatch.normalize.pipeline import Normalizer
from jobmatch.schemas import RawJob

normalizer = Normalizer.from_settings(load_settings())
raw = RawJob(
    source="manual", source_id="test-1", url="https://example.org/1",
    title="Analyste d'affaires (contrat de 6 mois)", company="Exemple Inc.",
    location="Montréal, QC",
    description="Contrat de 6 mois.\nExigences :\n- SQL\n- BPMN",
    fetched_at=datetime.now(UTC),
)
print(normalizer.normalize(raw, now=datetime.now(UTC)).model_dump_json(indent=2))
EOF
```

When you have new data

Real contractors

- Create `data/contractors.csv` and `data/contractor_facets.csv` with the same columns as the files in `data/samples/` . `data/` is git-ignored, so the real files stay on your machine.
- One row per contractor, one row per area of expertise (facet).
- Separate multiple values with `|` , e.g. `en|fr` , `CA-ON|CA-QC` .
- Run `validate-data` ; it names the file and row of every problem.
- From phase 4, run `embed-contractors` ; it re-embeds only facets whose text changed.

New job postings

- Today: test a single posting with the normalizer snippet above.
- Phase 3: `ingest-manual <file>` takes a CSV or JSON template, and pasted LinkedIn page text.
- Re-running an import is safe: jobs already stored (same source and ID) are skipped.

New skills or synonyms

- Edit `config/skills_taxonomy.yaml` : a lowercase skill name, then English and French aliases.
- Avoid everyday words as aliases (go, spring, tableau): use a longer phrase.

Wording, places, thresholds

- Contract or permanent wording: `config/contract_phrases.yaml`
- Cities and provinces: `config/canada_locations.yaml`
- Section headers, title noise, seniority

- Use the same skill names in `contractor_facets.csv` so overlap is counted.
- Run `validate-data` (catches an alias used twice), then `pytest`.

words: `config/normalization.yaml`

- Thresholds marked GUESS: `config/settings.yaml`; tune them once reviews exist.
- Changing the embedding model needs a decision and a full re-embed.

Contractor columns

FILE	COLUMN	FORMAT
`contractors.csv`	`contractor_id`	Any text, kept as text (e.g. `C001`); must be unique
	`display_name`	Personal data; never written to logs
	`profile_text_raw`	Free description of the person's expertise
	`languages`	Two-letter codes: `en|fr`
	`regions` · `remote_preference`	`CA-ON|CA-QC`, or `CA` for anywhere · onsite, hybrid, remote or any
	`noc_codes`	Five digits each, leading zeros kept: `00012|21231`
	`rate_min` · `rate_max` · `rate_unit` · `rate_currency`	Optional. Unit is hour, day, week, month or year; currency like `CAD`
`contractor_facets.csv`	`contractor_id` · `facet_id`	Links to a contractor; facet ID unique per contractor (e.g. `data_engineering`)
	`area`	Short name of the expertise area
	`skills` · `domains`	Skill names from the taxonomy, separated by `|` · sectors such as `banking|public sector`
	`years` · `seniority`	Number · junior, intermediate, senior, lead or executive

> Two habits protect the data. Close the live CSVs in Excel before a run (Excel locks files and rewrites formatting), and when saving from Excel use "CSV UTF-8" so accents survive. A cell left

> empty means no value.